

Jalur yang Benar

Soad 1

S	/	.	.	.
.	/	.	/	G
.	.	.	/	.
/	/	.	.	.
X	.	.	P	.

D(2,1) D(5,3)
D(3,1) R(5,4) P
R(3,2) R(5,5) OR U(4,4)
R(3,3) U(4,5) R(4,5)
D(4,3) U(3,5)
 U(2,5)

Soad 2

S	.	.	/	.	.
.	/	.	/	.	G
.	/	.	/	.	.
/	/	.	/	.	/
X	.	.	/	.	P
/	/	.	X	.	.

Way #1
R(1,2)
R(1,3) D(4,5) U(5,6)
D(2,3) D(5,5) U(4,6)
D(3,3) ★ D(6,5) R(4,7)
D(3,4) R(6,6) U(3,7)
R(3,4) R(6,7) P U(2,7)
R(3,5) L(6,6)
Way #2
D(2,1) R(4,2)
D(3,1) R(4,3)
D(4,1) U(3,3) ★ (same as way #2)

Soad 3

S	.	.	/	.	.	.
.	/	.	/	/	/	G
.	/	.	/	/	/	.
/	/	.	/	/	/	.
.	.	.	/	/	/	.
.	/	.	X	.	/	.
/	/	.	/	/	/	F
X	.	.	.	X	.	.

R(1,2) R(5,7) D(9,9)
R(1,3) D(6,7) L(9,8)
D(2,3) D(7,7) L(9,7)
D(3,3) D(8,7) U(8,7)
R(3,4) D(9,7) U(7,7)
R(3,5) R(9,8) U(6,7)
D(4,5) R(9,9) U(5,7)
D(5,5) U(8,9) P U(4,7)
R(5,6) U(3,7)

Some Credit (for study/look at and some code source):

<https://www.geeksforgeeks.org/dsa/a-search-algorithm/>

<https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/>

<https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/>

<https://medium.com/omarelgabrys-blog/path-finding-algorithms-f65a8902eb40>

https://www.reddit.com/r/algorithms/comments/kigv8u/is_there_a_mathematical_way_to_check_4_directions/

<https://stackoverflow.com/questions/4991638/how-to-implement-c-inequality-operators-for-aggregate-structs>

<https://www.geeksforgeeks.org/cpp/cpp-program-for-bfs-traversal/>

More Credit:

☐	10:45 PM		How to implement C++ (in)equality operators for aggregate structs? - Stack Overflow	stackoverflow.com	⋮
☐	10:44 PM		Offset between two array element addresses in C - Stack Overflow	stackoverflow.com	⋮
☐	6:33 PM		[C++] - Breadth-First Search on a 2D Array in C++ - SheCodes	shecodes.io	⋮
☐	6:33 PM		bfs code in c++ with 2d maze array - Google Search	google.com	⋮
☐	6:31 PM		Breadth-First Search in a 2D Matrix - DEV Community	dev.to	⋮
☐	6:31 PM		Implement BFS-based maze solver Meta Interview Question	prachub.com	⋮
☐	6:31 PM		PracHub: Real Interview Questions from Real Candidates	prachub.com	⋮
☐	6:28 PM		c++ - How to reconstruct the path of a 2d maze after using BFS to find the shortest path - Stack Overflow	stackoverflow.com	⋮
☐	6:28 PM		Breadth First Traversal (BFS) on a 2D array - GeeksforGeeks	geeksforgeeks.org	⋮
☐	6:25 PM		Breadth First Search or BFS for a Graph - GeeksforGeeks	geeksforgeeks.org	⋮
☐	6:25 PM		C++ Program for BFS Traversal - GeeksforGeeks	geeksforgeeks.org	⋮
☐	6:24 PM		Is there a mathematical way to check 4 directions on a grid? : r/algorithms	reddit.com	⋮
☐	6:24 PM		Path Finding Algorithms. BFS, DFS(Recursive & Iterative)... by Omar Elgabry OmarElgabry's Blog Medium	medium.com	⋮
☐	6:24 PM		Special Task Programming - Google Docs	docs.google.com	⋮
☐	6:24 PM		Depth First Search or DFS for a Graph - GeeksforGeeks	geeksforgeeks.org	⋮
☐	6:24 PM		A* Search Algorithm - GeeksforGeeks	geeksforgeeks.org	⋮
☐	1:48 PM		offset array for 4 direction - Google Search	google.com	⋮
☐	1:47 PM		Offset Array with defined rotation transforms - Grasshopper - McNeel Forum	discourse.mcneel.com	⋮
☐	1:46 PM		offset array - Google Search	google.com	⋮
☐	1:42 PM		BFS Graph Algorithm(With code in C, C++, Java and Python)	programiz.com	⋮

Map yang Digunakan

Soal 1:

#	#	#	#	#	#	#
#	S	#	.	.	.	#
#	.	#	.	#	G	#
#	.	.	.	#	.	#
#	#	#	.	.	.	#
#	X	.	.	F	.	#
#	#	#	#	#	#	#

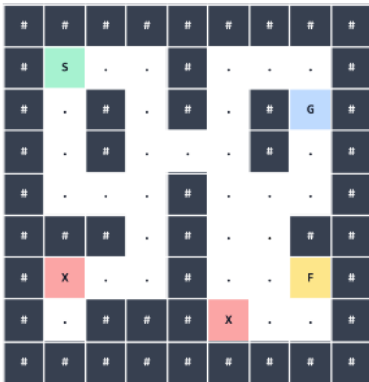
```

# # # # # # #
# S # . . . #
# . # . # G #
# . . . # . #
# # # . . . #
# X . . F . #
# # # # # # #

```

Simbol	Keterangan
#	Dinding - tidak dapat dilewati
S	Start - posisi awal robot
.	Jalur yang dapat dilewati
F	Flag - bendera yang harus diambil
X	Land mine - tidak boleh diinjak
G	Goal/Base - tempat membawa bendera

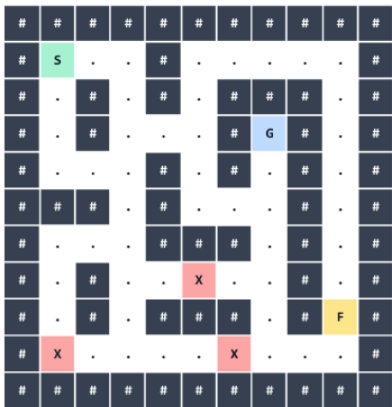
Soal 2:



```
# # # # # # # # # #
# S . . # . . . #
# . # . # . # G #
# . # . . . # . #
# . . . # . . . #
# # # . # . . # #
# X . . # . . F #
# . # # # # X . #
# # # # # # # # #
```

Simbol	Keterangan
#	Dinding - tidak dapat dilewati
S	Start - posisi awal robot
.	Jalur yang dapat dilewati
F	Flag - bendera yang harus diambil
X	Land mine - tidak boleh diinjak
G	Goal/Base - tempat membawa bendera

Soal 3:



```
# # # # # # # # # #
# S . . # . . . . #
# . # . # . # # # . #
# . # . . . # G # . #
# . . . # . # . # . #
# # # . # . . . # . #
# . . . # # # . # . #
# . # . . X . . # . #
# . # . # # # . # F #
# X . . . . X . . . #
# # # # # # # # # #
```

Simbol	Keterangan
#	Dinding - tidak dapat dilewati
S	Start - posisi awal robot
.	Jalur yang dapat dilewati
F	Flag - bendera yang harus diambil
X	Land mine - tidak boleh diinjak
G	Goal/Base - tempat membawa bendera

Penjelasan singkat

Program ini menggunakan algoritma OOP untuk menyelesaikan dan mencari jalur terpendek yang dapat dilewati pada maze dua dimensi.

Untuk program maze-nya, petanya disimpan dalam matriks array 2D (`vector<string>`) dan fungsi pencarian simbol (S, F, G) dilakukan secara otomatis.

Algoritma Path Finding yang digunakan adalah BFS (Breadth-First Search) untuk mencari cara terpendek / jumlah move minimum yang dilakukan dalam 2 tahap, yaitu saat S ke F dan saat F ke G, serta algoritma memeriksa 4 arah gerak (UP, DOWN, LEFT, RIGHT) dan memastikan koordinat berikutnya aman (tidak menembus dinding # atau menyetuh X).

Untuk mencetak pergerakan, setiap langkah robot dicatat ke dalam antrean (*queue*), dan program akan memperbarui posisi robot di setiap langkah, mengubah karakter *tile* aktif menjadi R, lalu mencetak ulang seluruh peta maze ke terminal.

Terakhir, program menghitung total moves, mencetak status Flag Captured (saat di F) dan Mission Complete (saat di G)